Informe de Laboratorio 08
Tema: PostgreSQL + CRUD + API REST + Fetch API

Nota

Estudiante	Escuela	Asignatura
Piero Franco Pachao Bellido. Kener Paniora Quispe. Claudio Cruz Huaraca. Arnold Choque Mendoza .	Escuela Profesional de Ingeniería de Sistemas	Introduccion al Desarrollo Web Semestre: II Código: 2025000

Laboratorio	Tema	Duración
08	PostgreSQL + CRUD + API REST + Fetch API	10 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	Del 21 de octubre 2026	Al 2 de noviembre 2026

1. Tarea

- Modelar e implementar una base de datos relacional en PostgreSQL ejecutando sentencias SQL estandarizadas.
- Conectar la aplicación Node.js/Express con PostgreSQL utilizando un Pool de Conexiones (driver pg) y consultas parametrizadas para prevenir Inyección SQL.
- Consumir los endpoints de la API RESTful desde la interfaz web cliente mediante la Fetch API (async/await) para completar la integración Full Stack.

2. Equipos, materiales y temas utilizados

- Visual Studio Code 1.137.0 .
- Thunder Client v2.41.3.
- Express.js, coleccion-postman-cursos.json, JavaScript.
- GitHub con cuenta institucional.
- Terminal de comandos .
- Git 2.55.0.windows.3
- Node.js version 24.21.0 y npm.

3. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- https://github.com/ppachao-cmyk/ctx.git

4. Antes de empezar.

4.1. Crear cuenta en supabase

Para poder realizar estos cambios y mejoras al proyecto se tiene que crear una cuenta en Supabase (el plan gratuito alcanzara) ademas se debe tener claro el modelo de datos (tablas y relaciones)

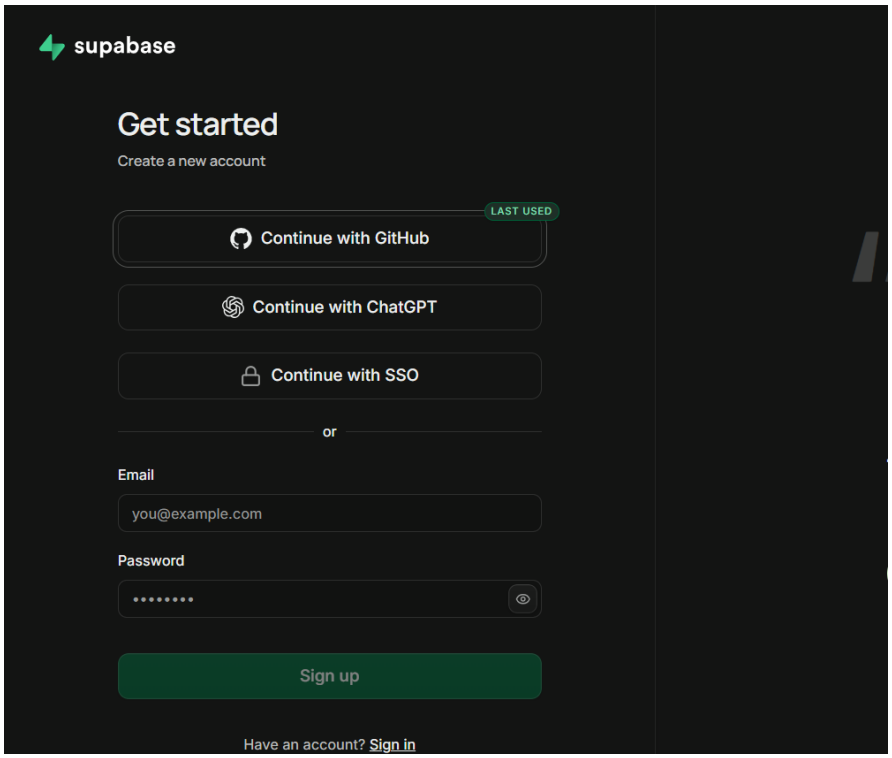


Figura 1: Crear cuenta supabase

5. Conectar un backend Node.js (Express) a Supabase

5.1. Paso 1: Crear el proyecto en Supabase

1. Entra a Supabase e inicia sesion.
2. Pulsa New project

3. Se debe llenar: Name (por ejemplo ctx), Database Password y Region (la mas cercana a los usuarios).
4. Pulsa Create new project y espera

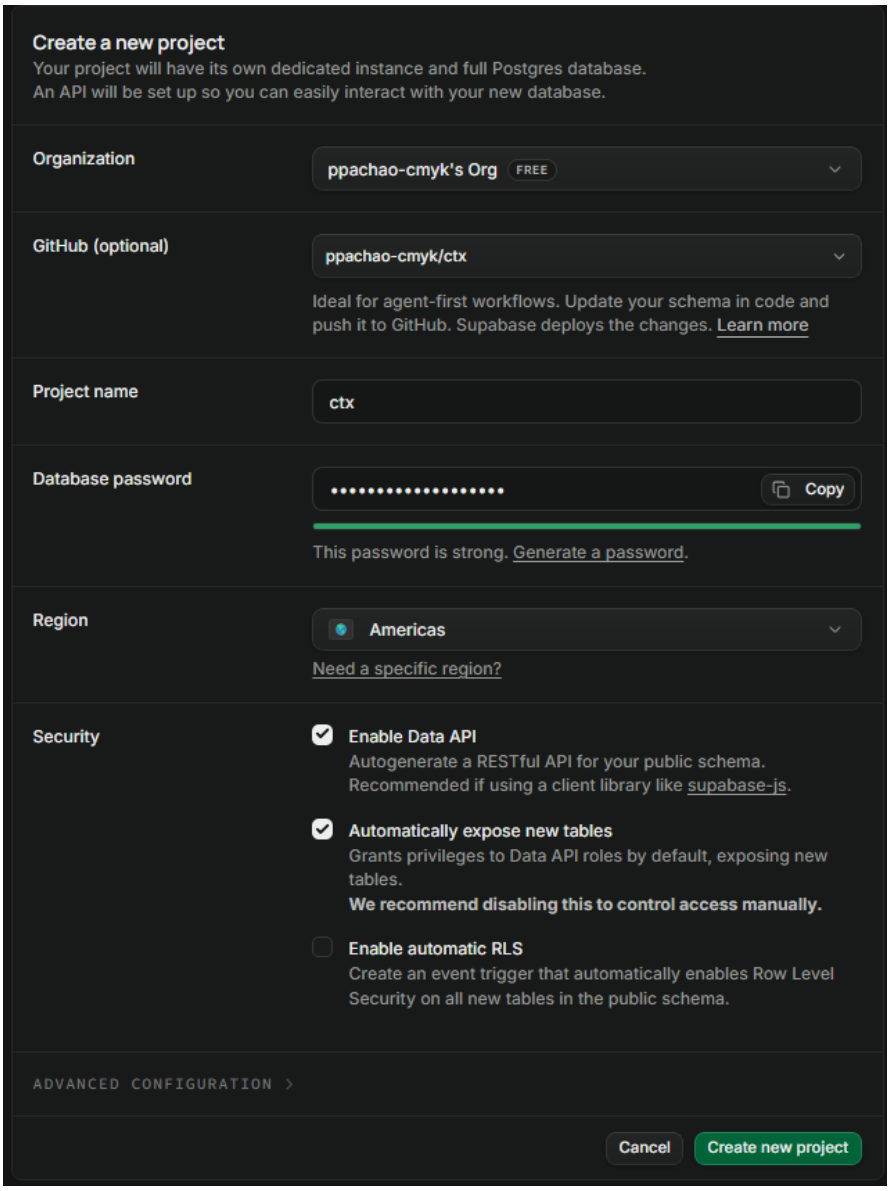


Figura 2: Crear el proyecto

5.2. Paso 2: Crear las tablas

1. Menu izquierdo: table Editor
2. Y creamos nuestras tablas de nuetros proyectos

5.2.1. Tabla usuario

Representa las personas registradas

Un mismo usuario puede vender muchos productos, comprar y opinar. Sus datos (nombre, email, clave) se guardan una sola vez.

Se relaciona con lo demas como: Un usuario vende muchos productos (producto.vendedor_id).

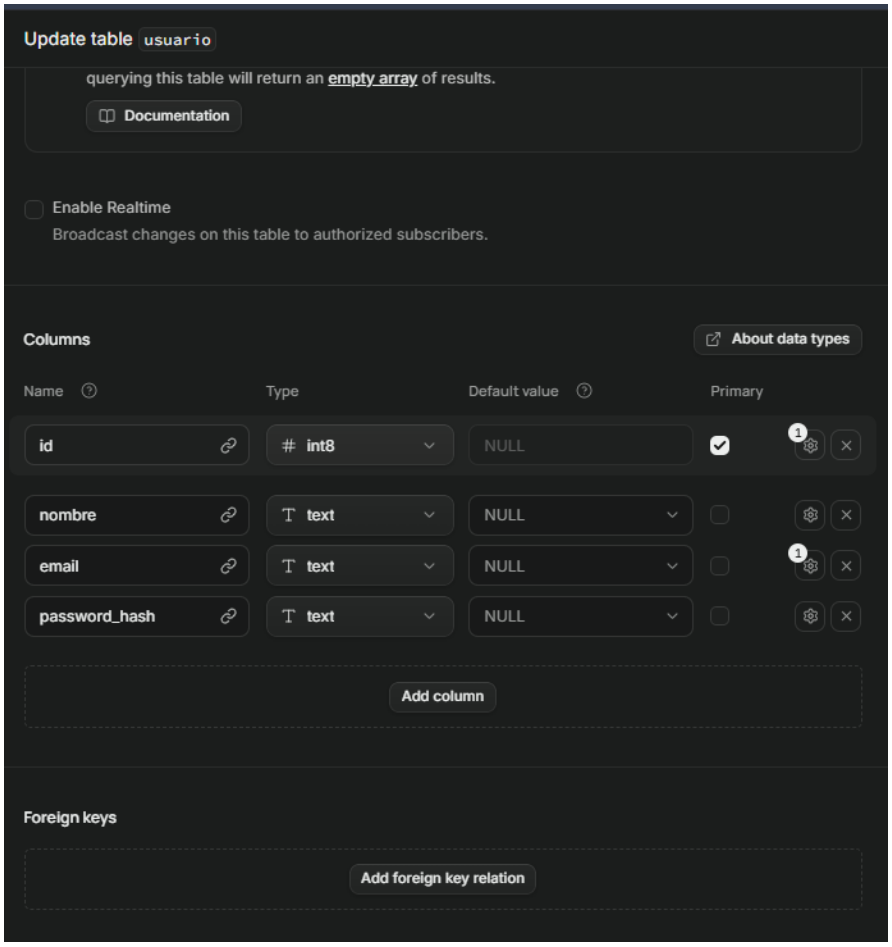


Figura 3: Configuracion del usuario

5.2.2. Tabla producto

- Representa lo que se publica a la venta
- Pertenece a un vendedor y cambia con el tiempo (precio, stock)
- Un producto tiene muchas compras y muchas resenas.

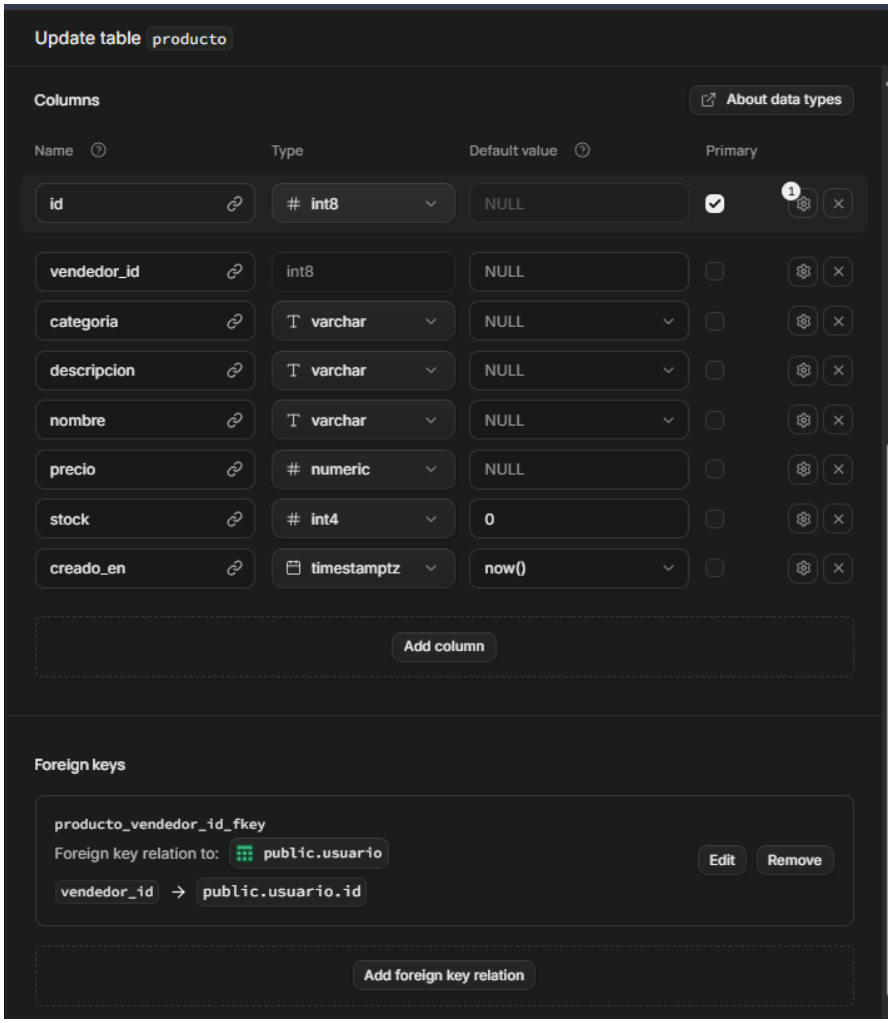


Figura 4: configuracion de producto

5.2.3. Tabla compra

- Representa cada operacion de compra
- Es un registro historico: debe quedar igual aunque el producto cambie o se borre.
- Una compra une a dos usuarios: el comprador y el vendedor (comprador_id, vendedor_id).

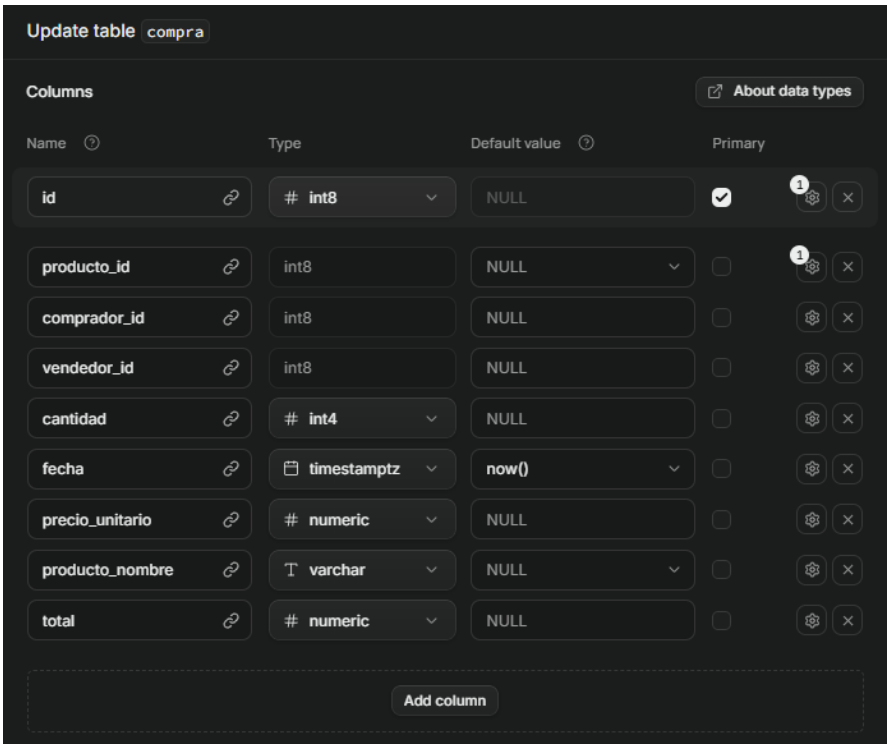


Figura 5: tipos

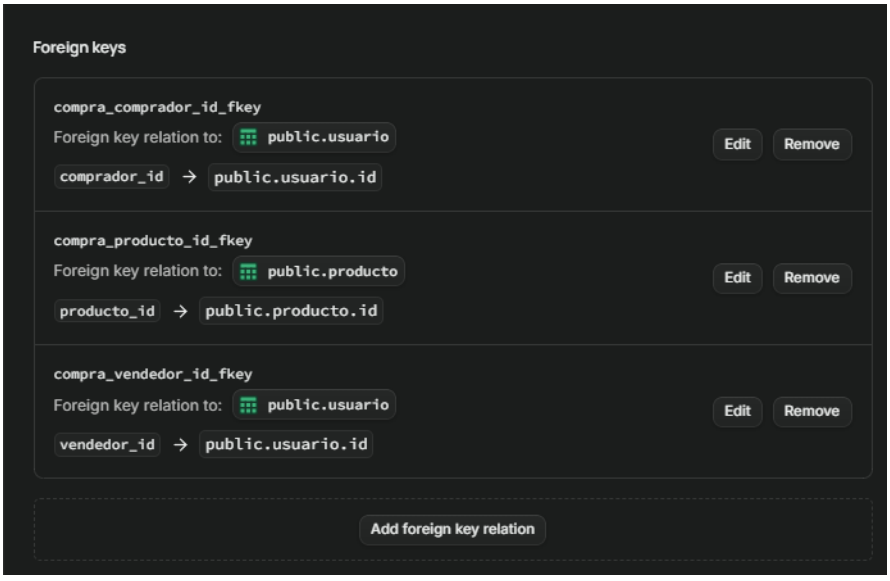


Figura 6: foreign key

5.2.4. Tabla reseña

- Representa la opinion de un usuario sobre un producto
- Un producto tiene muchas reseñas y un usuario escribe muchas, pero cada par usuario-producto

solo una.

- Una reseña une a un usuario con un producto, y la restriccion unique (producto_id, usuario_id) impide reseñar dos veces el mismo.

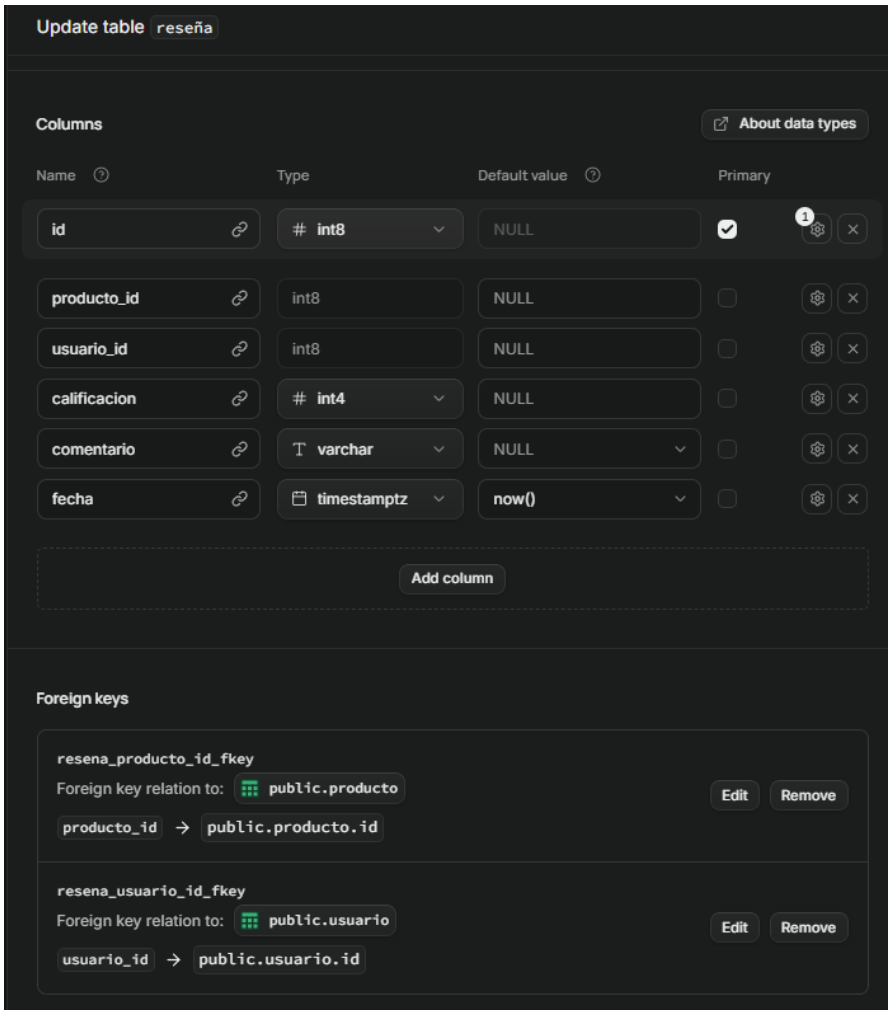


Figura 7: Enter Caption

5.2.5. Restricciones que acompanan a los tipos

- not null: el dato es obligatorio (un producto sin precio no tiene sentido).
- unique en email: dos personas no pueden registrarse con el mismo correo
- check: reglas de valor, como stock ¿= 0, cantidad ¿0, precio ¿0 y calificacion between 1 and 5.
- default now(): la base pone la fecha sola al crear la fila.
- on delete cascade o set null: que pasa con los datos hijos al borrar al padre. Las resenas se borran con su producto, pero las compras se conservan, porque son historial.

5.2.6. El por que esos tipos de datos

Tabla 1: Tipos de datos en bases de datos, ubicación y justificación

Tipo	Donde	Por que
`bigint con identity`	Todos los id	Números enteros que la base genera sola y nunca repite. `bigint` evita quedarse sin números. Las llaves foráneas usan el mismo tipo para poder enlazar.
`text`	nombre, email, password_hash, categoria	Texto sin límite natural. En PostgreSQL rinde igual que `varchar`.
`varchar(n)`	nombre de producto (80), descripcion y comentario (500), categoria (30)	Texto con tope, el mismo que valida tu backend. La base también lo hace cumplir.
`numeric(10,2)`	precio, precio_unitario, total	Decimal exacto con 2 decimales. Con `float`, sumas como $0{,}1 + 0{,}2$ dan resultados con error, inaceptable para dinero.
`integer`	stock, cantidad, calificacion	Enteros pequeños; no hay unidades "a medias".
`timestamptz`	fecha, creado_en	Fecha y hora con zona horaria, para que el momento sea el mismo sin importar desde donde se vea.

5.3. Paso 3: Copiar las credenciales

Se tiene que ir a proyect sentings y de ahi vas a API keys y compias lo siguiente

- Project URL: https://xxxxxxxx.supabase.co
- service_role o Secret key

5.4. Paso 4: Instalar dependencias

```
npm install @supabase/supabase-js cookie-session
npm uninstall express-session
npm install
```

- @supabase/supabase-js: la libreria oficial para hablar con Supabase desde Node.
- cookie-session: reemplaza a express-session. Este ultimo guardaba las sesiones en la memoria del servidor
- npm uninstall express-session: ya nadie lo usa. Quitarlo va evitar dependencias muertas y/o confusiones.

5.5. Paso 5: Crear/modificar el archivo .env

```
PORT=3000
SESSION_SECRET=Se puede escribir cualquier cosa
SUPABASE_URL=https://xxxxxxxx.supabase.co
SUPABASE_SERVICE_ROLE_KEY=Aqui se pone la clave que se tiene que usar
```

- SUPABASE_URL: la direccion de tu proyecto.
- SUPABASE_SERVICE_ROLE_KEY: la llave que te identifica ante Supabase.
- SESSION_SECRET: la frase con la que se firma la cookie de sesion.

5.6. Paso 6: Crear backend/config/supabase.js

```
const { createClient } = require('@supabase/supabase-js');

const { SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY } = process.env;

if (!SUPABASE_URL || !SUPABASE_SERVICE_ROLE_KEY) {
  throw new Error('Faltan SUPABASE_URL y SUPABASE_SERVICE_ROLE_KEY en el
    archivo .env');
}

module.exports = createClient(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, {
  auth: { persistSession: false, autoRefreshToken: false },
});
```

- Un solo archivo para el cliente: Node guarda en cache lo que se importa con require, asi que todos los controllers comparten la misma conexion en vez de crear una por consulta.
- Revisar las variables al inicio y lanzar un error claro: es preferible que el servidor falle al arrancar con "Faltan SUPABASE_URL..." que arrancar y fallar despues con un error confuso en medio de una peticion.

5.7. Paso 7: Borrar archivos innecesarios

Se tiene que borrar toda la carpeta data/ y tambien el archivo siguienteId.js

Los arreglos de data/ eran la base de datos temporal y siguienteId era su contador de ids; Supabase hace ambas cosas mejor, asi que esos archivos quedaron como piezas de un sistema que ya no existe.

6. Conectando a vercel

- Para desplegar la aplicación en un entorno de producción accesible a través de la web, se utilizó la plataforma Vercel, una infraestructura basada en Serverless Functions y alojamiento optimizado para aplicaciones Full Stack.

6.1. Paso 1: Importar el repositorio desde GitHub

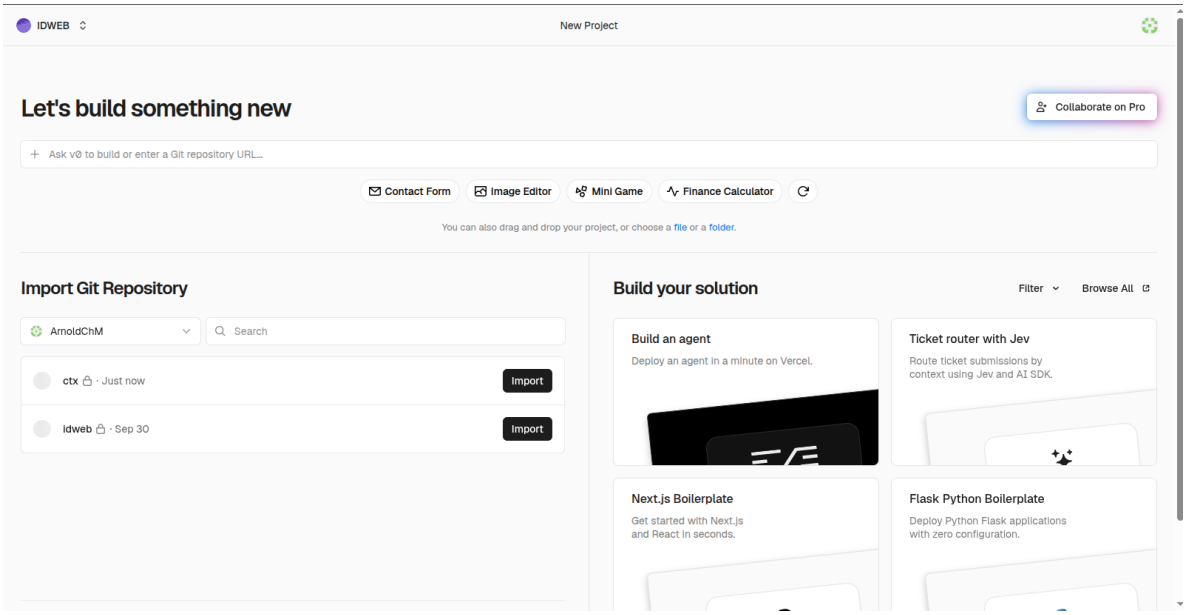


Figura 8: Enter Caption

- Iniciar sesión en la plataforma de Vercel en este caso vincule mi cuenta institucional de GitHub (ArnoldChM / ppachao-cmyk).
- En el panel principal (Dashboard), hacer clic en el botón Add New...
- En la sección Import Git Repository, buscar el repositorio del proyecto ctx (o ppachao-cmyk/ctx) y hacer clic en el botón Import.

6.2. Paso 2: Configuración del proyecto y variables de entorno

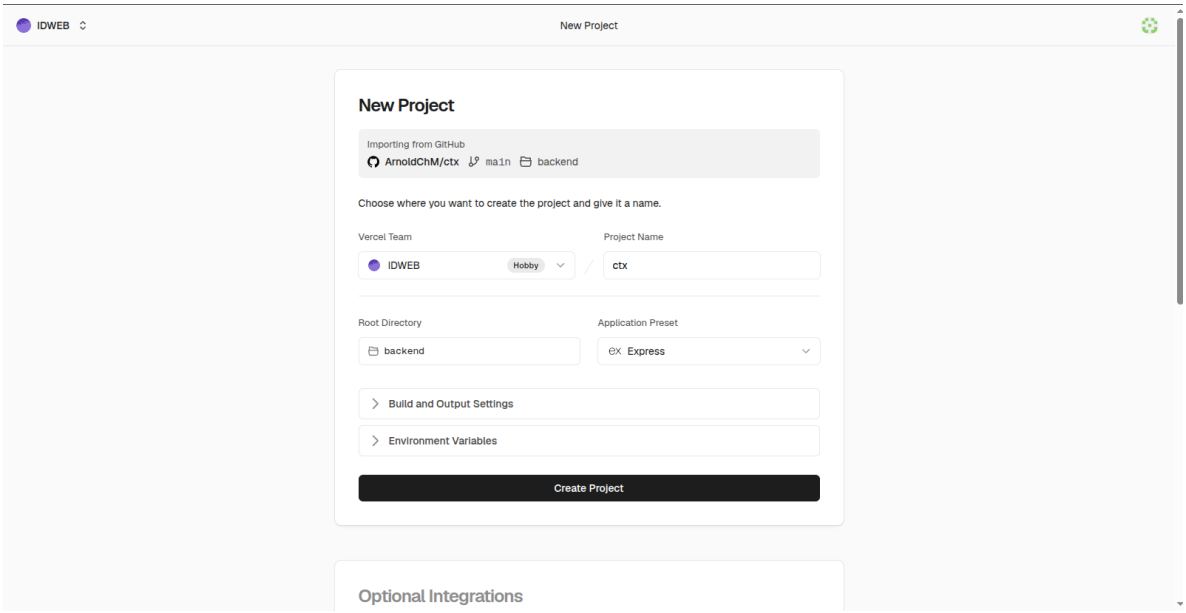


Figura 9: Enter Caption

- Se asignó el nombre del proyecto en Vercel (por ejemplo, ctx)
- Dado que la estructura del proyecto está dividida en subcarpetas dentro del repositorio monorepo, se configuró la carpeta backend como directorio raíz para la compilación del servidor (o la raíz del proyecto según el modelo de funciones).
- Se desplegó la pestaña Environment Variables para registrar de forma segura las credenciales configuradas previamente en el archivo .env

6.3. Paso 3: Despliegue del proyecto

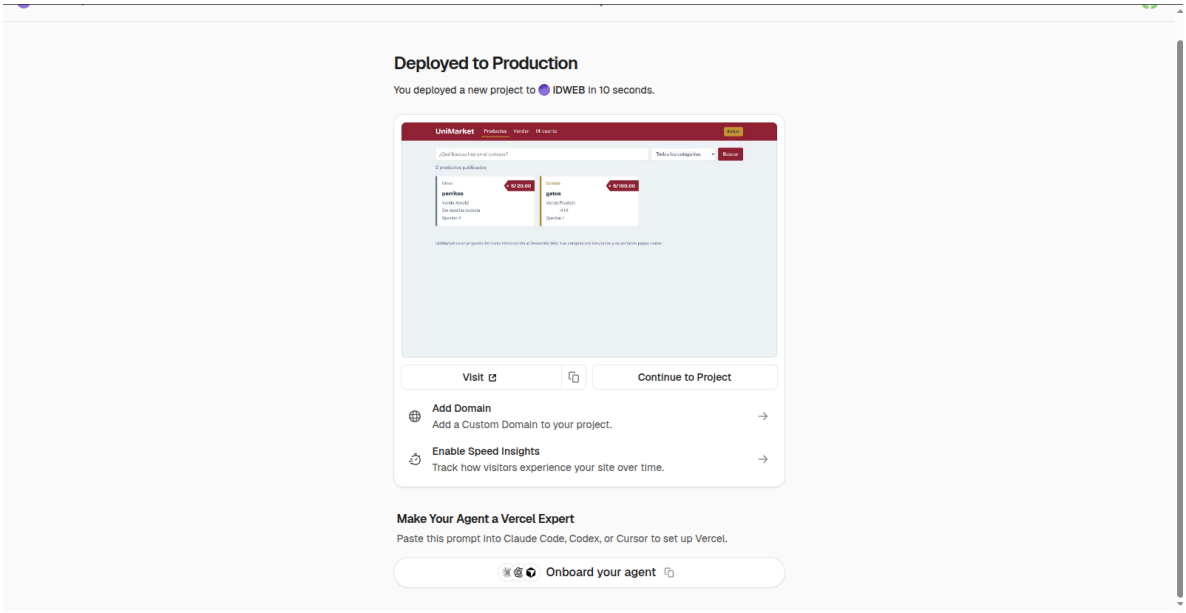


Figura 10: Enter Caption

- Tras validar las variables de entorno y el Root Directory, se presiona el botón Deploy.
- Finalizado el proceso en pocos segundos, aparece la pantalla Deployed to Production, indicando que la aplicación ha sido desplegada satisfactoriamente y generando un enlace de producción dominio .vercel.app.

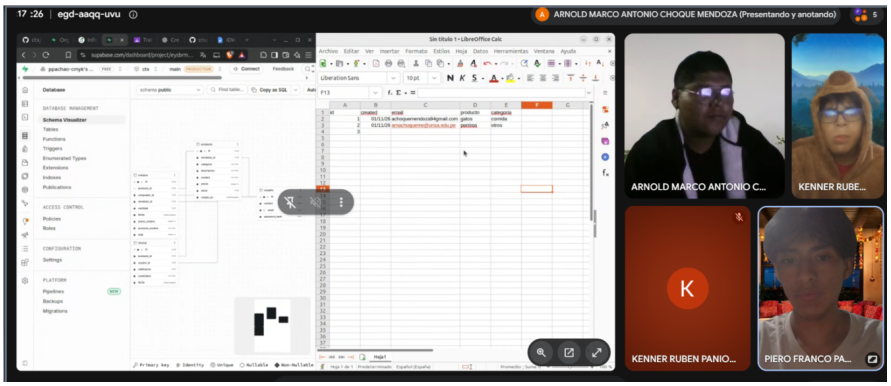


Figura 12: Evidencias

6.4. Paso 4: Verificación y pruebas del producto en producción

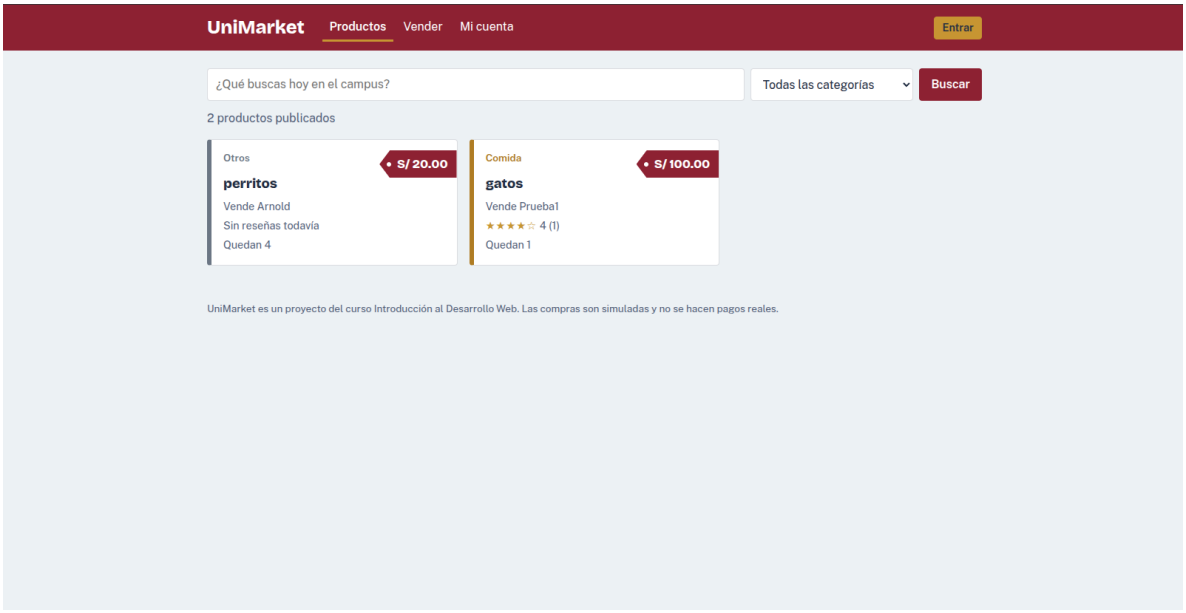


Figura 11: Enter Caption

- Se accede a la URL pública generada por Vercel para comprobar el funcionamiento del entorno cliente (UniMarket).

6.5. URL publica

- https://ctx-ten.vercel.app/

7. Evidencias de trabajo en equipo.

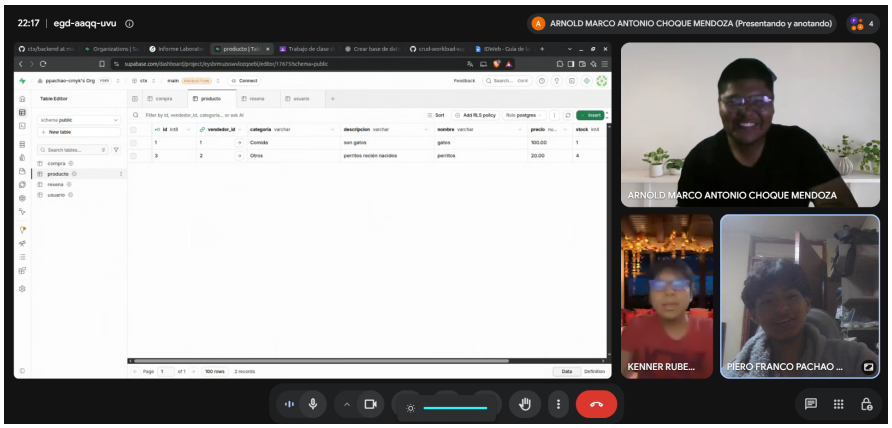


Figura 13: Evidencias

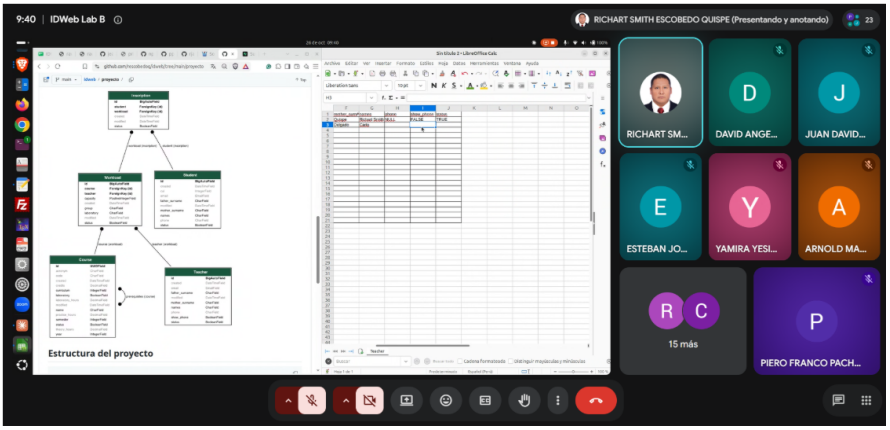


Figura 14: evidencia

8. Producto Final

8.1. Database

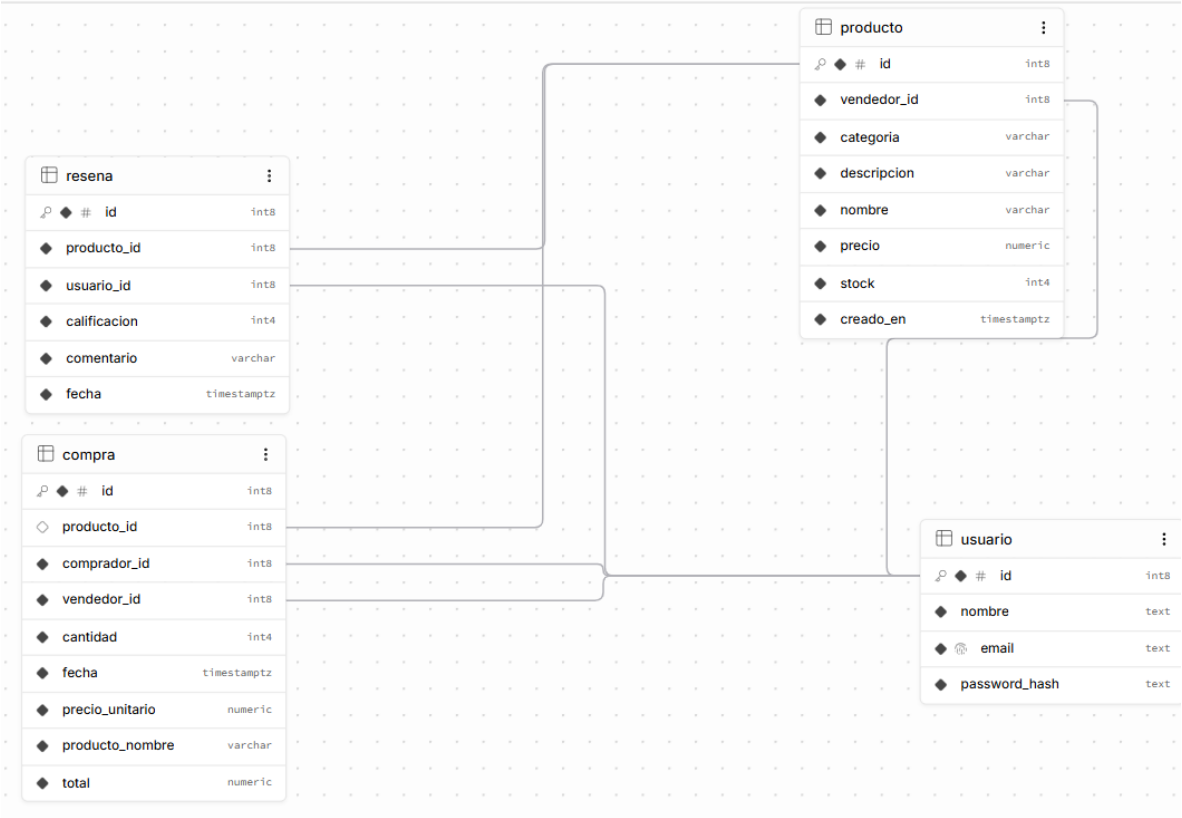


Figura 15: Creado en supabase

8.2. Estructura final

```
ctx/
|-- backend
|-- config/                     CONFIGURACION GLOBAL
|   |-- constantes.js           Valores fijos del sistema (ílmites, ícategoras,
    etc.)
|   |-- supabase.js             Cliente único de Supabase; valida las variables
    del .env
|-- controllers/
|   |-- authController.js       Registro, inicio y cierre de ósesin
|   |-- compraController.js     Registrar y consultar compras
|   |-- perfilController.js     Datos del usuario autenticado
|   |-- productoController.js   CRUD de productos
|   |-- resenaController.js     Crear y consultar ñreseas
|-- middlewares/
|   |-- authMiddleware.js       Verifica que haya ósesin activa; si no, bloquea el
                                acceso
|-- routes/
```

```
|   |-- authRoutes.js            Rutas de óautenticacin
|   |-- compraRoutes.js          Rutas de compras
|   |-- perfilRoutes.js          Rutas del perfil
|   |-- productoRoutes.js        Rutas de productos
|   |-- resenaRoutes.js          Rutas de ñreseas
|-- utils/
|   |-- aNumero.js               Convierte y valida únmeros recibidos como texto
|   |-- usuarioPublico.js        Devuelve el usuario sin datos sensibles
    (password_hash)
|-- .env                         Variables privadas: PORT, SESSION_SECRET, claves
    de Supabase
|-- .env.example                 Plantilla del .env sin valores reales
|-- .gitignore                   Excluye .env y node_modules del repositorio
|-- app.js                       Punto de entrada: crea el servidor, monta
    middlewares y rutas
|-- package-lock.json            Versiones exactas de las dependencias instaladas
|-- package.json                 Dependencias y scripts del proyecto
|-- README.md
|-- frontend
|   |-- .gitignore
|   |-- index.html
|   |-- pub
|        |-- js
|           |-- main.js
|        |-- style
|           |-- style.css
|   |-- index.html
|   |-- README.md
|-- .gitignore
|-- informes
|   |-- IDWeb_lab07_avance_proyecto.pdf
|   |-- IDWeb_lab08_avance_proyecto.pdf

|-- README.md
```

9. Preguntas

9.1. ¿Cómo ocurre exactamente una vulnerabilidad por Inyección SQL y por qué el uso de Consultas Parametrizadas (Prepared Statements) neutraliza completamente este vector de ataque?

Ocurre cuando el código concatena o interpola datos del usuario directamente en una consulta SQL. El atacante inyecta fragmentos de SQL maliciosos (por ejemplo ' OR '1'='1) que el motor interpreta como código, alterando la lógica de la consulta.

Las Consultas Parametrizadas (Prepared Statements) neutralizan el ataque porque:

- Separan la estructura SQL de los datos.
- Los valores se envían como parámetros independientes

- El motor de base de datos los trata siempre como datos literales, nunca como código SQL ejecutable.

9.2. Explique la ventaja técnica de utilizar un Pool de Conexiones (pg.Pool) frente a la creación e instanciación de una nueva conexión por cada petición HTTP individual.

Crear una conexión nueva a la base de datos por cada petición HTTP implica abrir una conexión de red, autenticarse y negociar parámetros, lo cual es lento y consume recursos.

Un Pool de Conexiones mantiene un conjunto de conexiones ya abiertas y listas. Cuando llega una petición, se toma una conexión libre del pool, se usa y se devuelve al terminar. Esto elimina el coste de crear y cerrar conexiones constantemente, mejora el rendimiento y permite soportar muchas peticiones concurrentes de forma eficiente.

9.3. ¿Qué es la política CORS (Cross-Origin Resource Sharing) en navegadores web y cómo se configura en Express para permitir solicitudes desde dominios o puertos distintos?

CORS (Cross-Origin Resource Sharing) es la política de seguridad de los navegadores que, por defecto, impide que una página web haga peticiones a un servidor que está en un dominio, puerto o protocolo diferente al de la propia página.

Para permitir solicitudes desde orígenes distintos en Express, se configura el middleware de CORS indicando qué dominios (o puertos) están autorizados. El servidor responde con las cabeceras HTTP necesarias (principalmente Access-Control-Allow-Origin) para que el navegador acepte la petición.

10. Rúbricas

10.1. Entregable Informe

Tabla 2: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

10.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. Persistencia y Consultas Parametrizadas	Conecta Node.js a PostgreSQL con Pool de Conexiones. Implementa todo el CRUD usando estrictamente consultas parametrizadas anti SQLi.	7	X	4	
2. Integración FrontEnd con Fetch API	Consume la API REST impecablemente con async/await y Fetch API. Actualiza el DOM cliente sin refresco de página ante cada operación CRUD.	7	X	5	
3. Integración FrontEnd con Fetch API	Diseño relacional sólido, tipos de datos óptimos, claves primarias y restricciones NOT NULL/UNIQUE correctamente especificadas.	3	X	2	
4. Cuestionario Teórico.	Responde acertadamente sobre Inyección SQL, mecanismo del Pool de conexiones y políticas CORS.	3	X	2	
Total		20		9	

11. Referencias

- PostgreSQL Global Development Group. (2025). PostgreSQL 16 Documentation. Postgresql.org.
- node-postgres Community. (2024). node-postgres (pg) Official Documentation. Node-postgres.com.
- MDN Web Docs. (2025). Using the Fetch API Y CORS. Developer Mozilla.